

Integrating SonarQube analysis with the Jenkins pipeline

To execute SonarQube analysis from an automated continuous integration pipeline we need the following:

1. Jenkins
2. Project repository (GitHub)
3. SonarQube project

Note: Since we have used the Java Maven sample application for this setup, it will work with a Maven application. For other applications, you can refer to the SonarQube official documentation.

If you have installed Jenkins locally, the Jenkins URL will be `http://localhost:8080/`. The following Jenkins plugins are required to be installed:

1. BlueOcean
2. GitHub
3. Maven Integration
4. Pipeline Maven Integration
5. Cobertura
6. SonarQube Scanner for Jenkins

To do so, go to *Jenkins home page* > *Manage Jenkins* > *Manage Plugins* > *Available* > *Search* for each plugin, and click *Install* without clicking the *Restart* button at the bottom of the page.

■ Creating a token in SonarQube:

1. Go to the SonarQube dashboard. Click on *My Account*.
2. Then go to *Security tab* > *Token section* > *Generate Tokens* section, give an appropriate name to the token, and click

on the *Generate* button to its right.

■ Creating Webhook in SonarQube for connection with Jenkins:

1. Go to *SonarQube dashboard* > *Administration* > *Configuration* > *Webhooks* and click the *Create* button on the right side to create a webhook.
2. Provide name, URL as 'JENKINS-URL/sonarqube-webhook/' and a secret for connection with Jenkins.
3. Click on the *Create* button to save.

■ Connecting SonarQube with Jenkins:

1. Go to *Manage Jenkins* > *Configure System* > *SonarQube servers*. Click on *Add SonarQube*.
2. Enter the name and the URL of the SonarQube server; for example, `https://test.sonarqube-url.com/`.
3. Server authentication token: Add the created token to Jenkins.
4. Select the *Secret* text from the drop down of *Kind*.
5. Copy and paste the generated token from SonarQube and provide an ID to a token. Click the *Add* button to save the token in Jenkins.
6. Select *Secret* from the drop down in SonarQube authentication token using the same ID.
7. Now click on the *Advanced* button to provide advanced settings.
8. Add the SonarQube webhook token to Jenkins just like we added the SonarQube authentication token as secret text.
9. Select the webhook secret from the drop-down in 'Webhook secret' for advanced configuration of *SonarQube Server* in Jenkins.
10. Save the configuration. It should look like what

The screenshot shows the Jenkins configuration page for SonarQube servers. The page is titled "Jenkins > configuration". Under the "SonarQube servers" section, there are several fields and options:

- Environment variables:** A checkbox labeled "Enable injection of SonarQube server configuration as build environment variables". Below it, a note states: "If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build."
- SonarQube installations:**
 - Name:** A text input field containing "SonarQube-Server".
 - Server URL:** A text input field containing "http://localhost:9000". Below it, a note states: "Default is http://localhost:9000".
 - Server authentication token:** A dropdown menu showing "SonarToken" and an "Add" button. Below it, a note states: "SonarQube authentication token. Mandatory when anonymous access is disabled."
 - Version of sonar-maven-plugin:** A text input field.
 - Webhook Secret:** A dropdown menu showing "SonarWebHook" and an "Add" button. A mouse cursor is pointing at this field.
 - Additional arguments:** A text input field.
 - Additional analysis properties:** A text input field.
- Trigger Exclusions (only for jobs with SonarQube as a post build action):** A text input field.

At the bottom of the page, there are "Save" and "Apply" buttons.

Figure 2: SonarQube server configuration to manage Jenkins